

YOGI VEMANA UNIVERSITY COLLEGE :: KADAPA
DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY

A MAJOR-PROJECT REPORT ON
REAL ESTATE PRICE PREDICTION USING MACHINE LEARNING

Submitted in partial fulfillment of the requirements for the award of degree in

MASTER OF COMPUTER SCIENCE

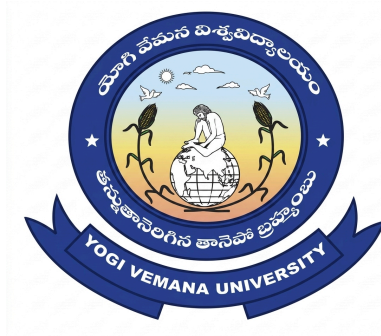
SUBMITTED BY
PINNINTI LAKSHMI NARAYANA
(00124509019)

Under the Guidance of
C.V. NARAYANA REDDY
ACADEMIC CONSULTANT
DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY



YOGI VEMANA UNIVERSITY, KADAPA – 516005
2024 – 2026

YOGI VEMANA UNIVERSITY COLLEGE :: KADAPA
DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY



CERTIFICATE

This is to certify that the project work entitled "**REAL ESTATE PRICE PREDICTION USING MACHINE LEARNING**" submitted in partial fulfillment of the requirements for the award of the degree of **Master of Science in Computer Science**, has been carried out by **PINNINTI LAKSHMI NARAYANA (00124509019)**, under my direct guidance and supervision during the academic year 2024–2026.

Project Coordinator

Project Guide

Head of the Department

External examiner

DECLARATION

I, **Pinniti Lakshmi Narayana**, a student of the Master of Computer Science programme at Yogi Vemana University, Kadapa, hereby solemnly declare that the project entitled "**Real Estate Price Prediction Using Machine Learning**" is the result of original research work carried out by me under the supervision of **C. V. Narayana Reddy**, Academic Consultant, Department of Computer Science and Technology.

I further declare that this project report has not been previously submitted to this or any other university or institution for the award of any degree or diploma.

I am aware that any misrepresentation of facts or submission of another person's work as my own constitutes academic dishonesty and may result in disciplinary action.

DATE:

PLACE: KADAPA

SIGNATURE OF THE CANDIDATE

PINNINTI LAKSHMI NARAYANA

(00124509019)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mention of the people, who made it possible whose constant guidance and encouragement crowned our efforts with success. We take this opportunity to express my deepest gratitude and appreciation to all those who have helped us directly or indirectly to word the successful completion of this project.

It is a great pleasure in expressing a deep sense of gratitude to our guide, **C. V. NARAYANA REDDY**, Academic consultant department of Computer Science and Technology (M.S.C) for her valuable guidance and thought-provoking discussion throughout the course of project work.

I take the opportunity to offer gratitude to our head of department of **Dr. K. SRINIVASA RAO**, Associate Professor of Computer Science and Technology (MSc) for providing all sorts of help during the project.

I take the opportunity to offer gratitude to our principal of the college **Prof T. SREENIVAS** for providing all sorts of help during the project work.

I express our thanks to all our college teaching and non-teaching staff members who encouraged and helped us in some way or other throughout the project work.

Finally, I am thankful to all our friends who have in some way or the other helped us in getting towards the completion of this project work

ABSTRACT

The real-estate sector plays a vital role in economic growth, and accurate property price prediction is essential for buyers, sellers, investors, and real-estate agencies. Traditional property valuation methods are often time-consuming, subjective, and heavily dependent on human expertise, which may lead to inaccurate estimations. To overcome these limitations, this project proposes a **Real- Estate Price Prediction System using Machine Learning** that provides accurate and data-driven price predictions.

The system utilizes historical real-estate datasets containing features such as location, area (in square feet), number of bedrooms and bathrooms, construction age, amenities, and nearby facilities. The dataset undergoes preprocessing steps including handling missing values, removing outliers, and normalizing data to improve model performance. Feature engineering techniques are applied to identify the most significant factors influencing property prices.

Multiple machine learning algorithms, including **Linear Regression, Decision Tree Regression, Random Forest Regression, and Gradient Boosting**, are implemented and compared to determine the most accurate model. These models are trained using supervised learning techniques and evaluated using performance metrics such as **Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2) score**. Among these, ensemble methods like Random Forest and Gradient Boosting provide higher accuracy by capturing complex non-linear relationships in the data.

The system is deployed with a user-friendly interface that allows users to input property details and receive instant price predictions. This improves decision-making, reduces dependency on manual estimation, and enhances transparency in the real-estate market.

In conclusion, the proposed system offers an efficient, accurate, and scalable solution for property price prediction. It can be further enhanced by integrating real-time market data, Geographic Information Systems (GIS), and advanced machine learning techniques for improved accuracy and wider applicability.

Keywords: *Real estate price prediction, Machine learning, Property valuation, Gradient Boosting, Random Forest, Data preprocessing, Regression models, Web application, Predictive analytics, Scikit-learn.*

INDEX

S.NO	CONTENT	PAGE NO.
1.	INTRODUCTION	1 - 5
1.1	EXISTING SYSTEM	2
1.1.1	KEY FUNCTIONALITIES	2
1.1.2	LIMITATIONS OF EXISTING SYSTEM	3
1.2	PROPOSED SYSTEM	3
1.2.1	ENHANCED FUNCTIONALITIES	4
1.2.2	TECHNOLOGIES STACK USED	5
1.2.3	ADVANTAGES OVER EXISTING SYSTEM	5
2.	LITERATURE SURVEY	6
3.	SYSTEM REQUIREMENTS SPECIFICATION	7 – 10
3.1	FUNCTIONAL REQUIREMENTS	7 - 9
3.2	NON-FUNCTIONAL REQUIREMENTS	9 - 10
3.3	HARDWARE REQUIREMENTS	10
3.4	SOFTWARE REQUIREMENTS	10
4.	SYSTEM DESIGN	11 - 26
4.1	INTRODUCTION	11
4.2	SYSTEM ARCHITECTURE	12
4.2.1	FRONT END	12
4.2.2	BACK END	13
4.3	UML DIAGRAMS	14
4.3.1	USE CASE DIAGRAM	15 - 16
4.3.2	CLASS DIAGRAM	16-20

4.3.3	SEQUENCE DIAGRAM	20-22
4.3.4	ACTIVITY DIAGRAM	23 – 26
5.	IMPLEMENTATION	27 - 30
5.1	TECHNOLOGY STACK	27 - 30
5.2	SAMPLE CODE	31 - 43
6.	TESTING	44 - 45
7.	DEPLOYMENT	46 - 47
8.	OUTPUT SCREENS	48 - 49
9.	CONCLUSION	50 - 51
10.	REFERENCES	52

LIST OF FIGURES

S.NO	FIGURES	PAGE NO.
1	4.2 SYSTEM ARCHITECTURE	13
2	4.3.1 USE CASE	16
3	4.3.2 CLASS DIAGRAM	20
4	4.3.3 SEQUENCE DIAGRAM	22
5	4.3.4 ACTIVITY DIAGRAM	26
6	8.1 HOME PAGE	
7	8.2 PREDICTION RESULT	48
8	8.3 PREDICTION WITH AMENITIES ENABLED	49

1.INTRODUCTION

In today's fast-paced world, accurate real-estate decision-making is essential for financial growth and stability. However, traditional property valuation methods are manual, time-consuming, and often inaccurate due to the complexity of factors such as location, area, and market trends. This creates the need for an intelligent and automated system to estimate property prices efficiently.

To address this, the **Real-Estate Price Prediction System using Machine Learning** is developed as a web-based platform that predicts property prices based on key features like location, area, and amenities. The system uses advanced machine learning models such as **Random Forest and Gradient Boosting** to analyze data and provide accurate predictions.

The platform allows users to enter property details and instantly receive predicted prices through a simple interface. This improves decision-making, reduces financial risks, and provides a faster and more reliable alternative to traditional methods.

MOST IMPORTANT GOALS OF THE SYSTEM

- **Accurate Property Price Prediction**
Provides reliable and data-driven price estimates using machine learning algorithms.
- **Improved Decision Making**
Helps buyers, sellers, and investors make informed financial decisions based on predicted values.
- **Automation of Valuation Process**
Eliminates manual effort and reduces dependency on human estimation methods.
- **Scalability and Efficiency**
Supports large datasets and delivers fast predictions through an optimized system.
- **User-Friendly Experience**
Offers a simple and interactive web interface for easy input and result visualization.

1.1 Existing System

The existing system for real-estate price estimation often relies on traditional methods such as manual valuation by real-estate agents, fixed pricing strategies, or basic online property listings that provide only approximate values. These traditional methods lack advanced features like automated price prediction, detailed feature analysis, visual insights, or real-time data processing, making it difficult for users to accurately estimate property values and understand market trends. There are also limited platforms that combine user-friendly interfaces with powerful machine learning models and analytical tools specifically designed for accurate and efficient real-estate price prediction.

1.1.1 Key Functionalities

1. User Input Interface:

Users can access the system through a web interface and provide property details such as location, area (square feet), number of bedrooms (BHK), bathrooms, and other relevant features required for price prediction.

2. Data Preprocessing:

The system processes the input data by handling missing values, encoding categorical data, and normalizing features to ensure accurate predictions.

3. Price Prediction:

The system uses machine learning algorithms such as Linear Regression, Random Forest, and Gradient Boosting to predict property prices based on the given inputs.

4. Model Evaluation:

The system evaluates model performance using metrics like Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared score to ensure prediction accuracy.

5. Result Display:

The predicted property price is displayed to the user instantly in a clear and understandable format through the interface.

6. Admin / System Management:

The system allows management of datasets, updating models, and monitoring overall system performance to maintain accuracy and efficiency.

1.1.2 Limitations of the Existing System

- Traditional manual methods (real-estate agent estimation, basic property listings, Excel- based analysis) are prone to errors and require significant time and effort.
- Existing platforms provide only approximate property prices without detailed feature-based analysis or advanced prediction models.
- Lack of visual analytics makes it difficult to understand the impact of factors like location, area, and amenities on property prices.
- No real-time prediction system to instantly estimate property prices based on user inputs.
- No integration of machine learning techniques for accurate, data-driven price estimation.
- Inflexibility in traditional systems that do not support automated processing, scalability, or user-friendly web-based access across devices.

1.1.3 Technology Stack Used in the Existing System

- **Front-End:** Basic HTML, CSS, and simple web interfaces used in property listing websites
- **Back-End:** N/A (manual processing or static data handling without machine learning models)
- **Platform:** Desktop-based or web-based platforms (property listing websites, Excel sheets, or manual record-keeping systems)

1.2 Proposed System

The proposed Real-Estate Price Prediction System using Machine Learning is a dynamic, user-friendly, and intelligent web-based application designed to address the limitations of traditional property valuation methods. Unlike conventional approaches that are often time-consuming, inconsistent, and dependent on manual estimation, this platform provides an accurate, efficient, and data-driven way for users to predict property prices based on multiple influencing factors.

This system emphasizes accuracy, automation, data analysis, and informed decision-making. By leveraging machine learning algorithms and advanced data processing techniques, the platform

offers a smarter solution for estimating property values, helping users understand market trends, reduce risks, and make better real-estate investment decisions.

1.2.1 Enhanced Functionalities of the Proposed System

1. Complete Property Data Management: ○ Users can input, update, and manage property details such as location, area, BHK, amenities, and other relevant features.

○ Feature-wise organization ensures better analysis and accurate price prediction.

2. User Role Classification:

○ The system defines roles such as Admin and User, each with specific functionalities.

○ Admins can manage datasets, monitor system performance, and maintain model accuracy.

3. Secure Access and Data Handling:

○ The system ensures secure handling of user inputs and prediction data.

○ Proper validation and session handling protect system integrity and reliability.

4. Interactive UI & Responsive Design:

○ Built with a user-friendly interface for easy input and result visualization.

○ Dashboard elements and simple navigation provide a smooth experience across devices.

5. Advanced Filtering and Smart Input Handling:

○ Users can input and analyze property details based on location, size, and features.

○ Structured input handling ensures efficient and accurate data processing.

6. Data Visualization and Insights:

○ Graphs and charts display relationships between features and property prices.

○ Visual insights help users understand real-estate trends and patterns.

7. Real-Time Price Prediction:

○ The system provides instant price predictions based on user input. ○ Machine learning models ensure fast and accurate results for better decision-making.

8. Model Processing and Analysis:

- The system applies algorithms like Random Forest and Gradient Boosting for prediction.
- Efficient processing ensures reliable outputs even with complex datasets.

9. Result Output and Reporting:

- Predicted prices are displayed clearly with relevant details.
- Users can analyze results for better property evaluation and investment planning.

1.2.2 Technology Stack Used

- **Frontend:** HTML, CSS, JavaScript (for user interface and input forms)
- **Backend:** Python (Flask / Django for handling requests and integrating machine learning models)
- **Database:** CSV / Dataset files or MySQL (for storing property data if required)
- **Machine Learning Libraries:** Scikit-learn, NumPy, Pandas (for data preprocessing, model training, and prediction)
- **Visualization Libraries:** Matplotlib or Seaborn (for data analysis and graphical insights)

1.2.3 Advantages over the Existing System

- Introduces accurate and automated property price prediction using machine learning models instead of manual estimation.
- Utilizes data-driven techniques to improve prediction accuracy and reduce human errors in property valuation.
- Offers a simple and user-friendly interface for entering property details and viewing predicted prices instantly.
- Provides better insights through data analysis and visualization of factors affecting property prices.
- Enables real-time prediction based on user inputs, improving speed and decision-making.
- Eliminates dependency on real-estate agents or approximate listings by providing reliable predictions.
- Scalable system with flexibility for future enhancements such as real-time data integration, GIS mapping, and advanced AI models.

2.LITERATURE SURVEY

In the paper titled **"Real Estate Price Prediction using Machine Learning"** by Tanguturu Akhil et al., published in the International Journal of Current Science (2023), the authors developed a system to predict house prices using machine learning techniques. The system utilizes features such as location, property size, number of rooms, and amenities, and applies algorithms like Linear Regression and Decision Tree. The study focuses on data preprocessing, model training, and evaluation using metrics such as Mean Squared Error (MSE) and R-squared values.

This literature emphasizes the importance of machine learning models in accurately predicting real-estate prices and assisting users in making informed decisions.

In the study titled **"Residential Property Price Prediction Using Machine Learning: MakanSETU"** by Yash Panchal et al., the authors propose a system that uses advanced algorithms such as Random Forest, Gradient Boosting, and Artificial Neural Networks. The system provides a web-based platform where users can input their requirements and receive suitable property suggestions. The study highlights the growing importance of real-estate prediction systems in handling large datasets and improving pricing accuracy.

This work contributes to understanding how machine learning and web technologies can improve real-estate decision-making and provide better investment insights.

In the paper titled **"Real Estate Price Prediction using Supervised Learning"** by Vedang Matey et al., published by the International Institute of Information Technology (2022), the authors analyze different supervised learning algorithms such as Linear Regression, Lasso Regression, and Decision Trees. The study focus on comparing model performance to identify the most accurate approach for predicting house prices.

This literature demonstrates the effectiveness of supervised machine learning techniques in improving prediction accuracy and highlights the need for efficient and scalable real-estate prediction systems.

3. SYSTEM REQUIREMENTS SPECIFICATION

The system requirement specification includes four categories.

- Functional Requirements
- Non – Functional Requirements
- Hardware Requirements
- Software Requirements

3.1 FUNCTIONAL REQUIREMENTS

1. User Input and Access

- The system shall allow users to access the application through a web interface.
- The system shall allow users to enter property details such as location, area, BHK, and amenities.
- The system shall validate user inputs before processing.
- The system shall allow users to perform multiple predictions as required.

2. Role-Based Access

- The system shall provide separate access and functionalities for:
 - ○ Admin
 - ○ User
- Admin shall have control over datasets, models, and system configurations.

3. Property Data Management

- The system shall allow users to input property details such as:
 - Location
 - Area (square feet)
 - Number of bedrooms (BHK)
 - Other relevant features
- The system shall process and use this data for prediction.
- The system shall allow modification of input values before prediction.

4. Dashboard Overview

- The system shall display a dashboard showing:
 - Entered property details
 - Predicted property price
 - Summary of analysis results

5. Data Preprocessing

- The system shall preprocess input data by handling missing values.
- The system shall encode categorical data such as location.
- The system shall normalize data for better model performance.

6. Prediction Functionality

- The system shall apply machine learning models to predict property prices.
- The system shall support models such as Linear Regression, Random Forest, and Gradient Boosting.
- The system shall provide accurate predictions based on input data.

7. Real-Time Processing

- The system shall generate predictions instantly after user input.
- The system shall ensure fast response time for better user experience.

8. Result Display and Visualization

- The system shall display predicted prices clearly to the user.
- The system shall provide basic visual insights (charts/graphs) for analysis.
- The system shall help users understand how features affect price.

9. Admin Panel

- The system shall allow admin to:
 - Manage datasets used for training models
 - Monitor system performance
 - Update or retrain machine learning models when required

3.2 NON – FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes of the system. They are not related to specific functions but are essential for the overall performance, usability, and reliability of the platform.

1. Usability

- The platform should provide a clean, intuitive, and user-friendly interface.
- Navigation between pages should be smooth and consistent across all devices.
- Property data input and price prediction processes should be simple and accessible even for non-technical users.

2. Performance

- The system should generate predictions within 2–3 seconds on average systems.

- The system should handle multiple users simultaneously without performance issues.
- Data processing and model execution should be optimized to reduce delays in prediction results.

3. Scalability

- The platform should be scalable to support an increased number of users and large datasets without affecting performance.
- Future modules (e.g., real-time data integration, advanced ML models, or GIS-based analysis) should be easy to integrate.

4. Security

- User inputs should be validated to prevent incorrect or malicious data entry.
- The system should ensure safe handling and processing of data.
- Unauthorized access to admin functionalities should be restricted through proper access control.

5. Maintainability

- The codebase should follow clean coding practices to allow easy updates or debugging.
- Proper comments and modular structure should make it maintainable for future developers.

3.3 HARDWARE REQUIREMENTS

- Processor : Intel Core i3
- RAM : Minimum 4 GB (8 GB recommended)
- Storage : Minimum 256GB or Greater
- Internet Connection : Required for dataset download, model updates, and web deployment

3.4 SOFTWARE REQUIREMENTS

- Operating System : Windows 11 or higher
- Software Environment : Visual Studio Code (VS Code)
- Programming language : Python
- Machine Learning Libraries : Scikit-learn, NumPy, Pandas
- Visualization Libraries : Matplotlib, Seaborn
- Framework : Flask (for web application)
- API Testing Tool : Postman (for testing APIs if deployed)
- Package Manager : pip (Python Package Manager)

4.SYSTEM DESIGN

4.1 INTRODUCTION

System design is a crucial stage of software development, which establishes a system's architecture, components, modules, interfaces, and data flow. It provides a blueprint for how the overall system will function, ensuring that all parts work together efficiently to meet the project's objectives.

In this project, **Real-Estate Price Prediction System using Machine Learning**, system design plays a vital role in enabling users to interact effectively with the platform and obtain accurate predictions. The system is structured using a modular architecture, where the front end collects user inputs and the back end processes the data using machine learning models to generate predictions.

This platform allows:

- Users to input property details such as location, area, BHK, and other features.
- The system to preprocess data and apply machine learning algorithms for prediction.
- Users to receive instant property price predictions.
- Users to analyze results for better real-estate decision-making.

The system is designed to ensure:

- A user-friendly interface for easy data input and result visualization.
- Efficient data processing and accurate prediction using machine learning models.
- Reliable backend processing for handling data and generating predictions.

The entire system can run both locally and on cloud environments, with deployment options using frameworks like Flask/Django for backend and web interfaces for frontend, providing an efficient, scalable, and real-time prediction system.

By designing the system in a modular and scalable way, it ensures better maintainability, efficient data handling, and improved user experience for all users.

4.2 SYSTEM ARCHITECTURE

System architecture refers to the fundamental structure of a software system, outlining its components, interactions, and design principles. It defines how different modules or components of the system are organized and how they communicate with each other to achieve the system's objectives. This includes the arrangement of hardware and software elements, data flow, interfaces, and system behavior. A well-designed system architecture ensures scalability, flexibility, maintainability, and reliability of the software system.

In this system architecture of the **Real-Estate Price Prediction System using Machine Learning**, the process is divided into two main parts: the Front End and the Back End, both working together to provide accurate and efficient property price predictions.

4.2.1 Front End (Client Side)

The front end is what the user sees and interacts with. It is developed using HTML, CSS, and JavaScript (or simple frameworks), serving as the user interface for entering property details and viewing predicted prices.

The front end allows users to:

- Users interact with the system through a web interface.
- Enter property details such as location, area (square feet), BHK, and other features.
- Modify input values and perform multiple predictions.
- View predicted property prices instantly.
- Visualize results through simple charts or graphical outputs.
- Navigate easily through input forms, results, and analysis sections.

The front end is responsible for:

- Sending user input data to the backend.
- Displaying the predicted results received from the backend.
- Sending HTTP requests to the backend using APIs.
- Presenting results in a clear and understandable format.
- Managing basic validations for user input.

4.2.2 Back End (Server Side)

The backend is developed using Python and machine learning libraries, connected to datasets or databases for prediction.

Python and Machine Learning Models:

- Python is used as the core programming language for implementing the system.
- Machine learning models such as Linear Regression, Random Forest, and Gradient Boosting are used for prediction.
- The system processes input data and applies trained models to generate results.
- Data preprocessing techniques such as encoding, normalization, and cleaning are applied.
- The system validates input data to ensure accuracy and consistency.

Dataset :

- Acts as the source of data for training and prediction.
- Stores property-related data such as location, area, BHK, and price.
- Supports structured data handling for model training and testing.
- Enables scalability for future data expansion and improvements.

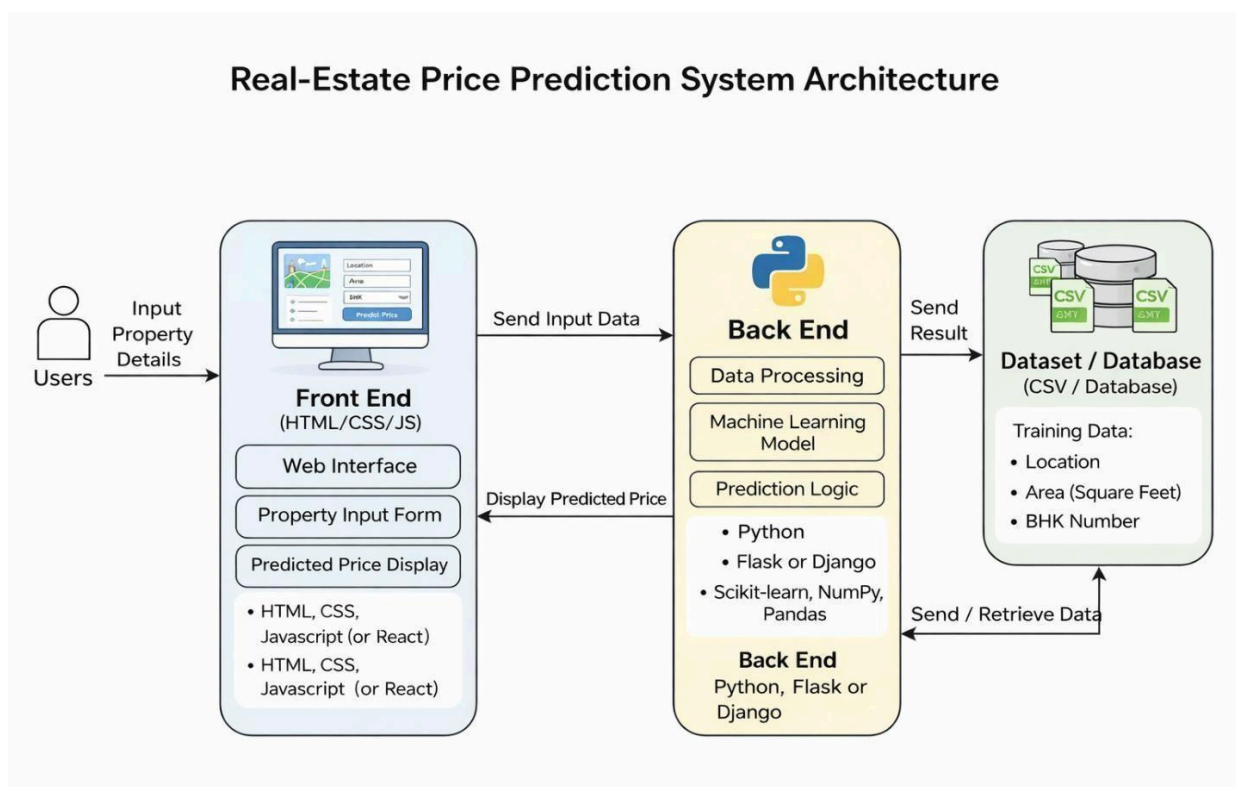


Fig.
4.2

4.3 UML DIAGRAMS

The **Unified Modeling Language (UML)** is a powerful tool used by software engineers to visualize, design, and model complex software systems. It provides a standardized way to represent different aspects of a system using various diagrams and structured notations. As software systems become more complex, UML helps developers understand system behavior, reduce errors, and improve development efficiency.

UML is not a development methodology but a modeling language that offers different types of diagrams to represent system structure and functionality. These diagrams act as visual representations, making it easier for developers, designers, and stakeholders to understand how the system works and how different components interact with each other. It also improves communication among team members and helps in faster development and debugging.

In the **Real-Estate Price Prediction System using Machine Learning**, UML diagrams are used to represent the system architecture, data flow, and interactions between different modules such as user input, data processing, machine learning model, and result generation.

UML diagrams provide a clear understanding of the workflow of the system and help in designing an efficient and scalable solution.

Some of the important UML diagrams used in this project are:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram

4.3.1 USE CASE DIAGRAM

A use case diagram is a visual representation of the interactions between a system and its users, illustrating the various ways users interact with the system to achieve specific goals or tasks. It consists of actors, use cases, and the relationships between them.

Fig. 4.3.1 Use case diagram

4.3.2 CLASS DIAGRAM

A class diagram in software engineering is a visual representation of the structure and relationships among classes in a system. It provides a high-level view of the system's architecture, showing how different classes interact with each other along with their attributes and methods.

Class diagrams are essential for designing and understanding object-oriented systems.

In a class diagram, each class is represented as a box divided into three sections: class name, attributes, and methods. Relationships between classes show how they interact, such as association and dependency.

Class diagrams help developers understand system structure, improve design, and ensure smooth development. They act as a blueprint for building the system.

Class Diagram For Real-Estate Price Prediction System

Classes:

1. Data set Generator Class

Represents the module responsible for generating and storing dataset.

Attributes:

- cities: list – List of cities
- num_rows: int – Number of records

Methods:

- generate() – Generates dataset
- save_csv() – Saves dataset into CSV file

2. Housing Record Class

Represents individual property records used in dataset.

Attributes:

- city: str – Property location
- state: str – State name
- area_sqft: int – Area of property
- bhk: int – Number of bedrooms
- amenities: dict – Property features
- price_lakhs: float – Property price
- is_new: bool – Property type

Methods:

- get Details() – Returns property details

3. Model Trainer Class

Represents machine learning model training module.

Attributes:

- model: Gradient Boosting – ML model
- encoders: dict – Data encoders

Methods:

- train() – Trains model
- evaluate() – Evaluates model
- save_model() – Saves trained model

4. PredictionRequest Class

Represents user input data for prediction.

Attributes:

- city: str – Input location
- area: float – Property area
- bhk: int – Number of bedrooms
- amenities: dict – Features

Methods:

- validate() – Validates input

5. PredictionResponse Class

Represents predicted output.

Attributes:

- price: float – Predicted price
- price_str: str – Formatted price
- price_per_sqft: int – Price per square foot
- city, state: str – Location

Methods:

- display() – Displays prediction result

6. FlaskApp Class

Represents backend system handling prediction.

Attributes:

- model: pkl – Saved model
- encoders: pkl – Encoders

Methods:

- index() – Handles homepage
- predict() – Predicts price
- load_model() – Loads trained model

Relationship:

In the **Real-Estate Price Prediction System using Machine Learning**, the Admin oversees core functions such as managing datasets, training models, and monitoring system performance. The User class represents individuals who interact with the system to input property details and obtain predicted prices.

Users can enter and modify property details such as location, area, BHK, and amenities to generate price predictions. They can view predicted results and analyze outputs for better decision-making. The Prediction Request class handles user input data, while the PredictionResponse class provides the predicted results generated by the system.

The Data set Generator class is responsible for creating and storing property datasets, and the Housing Record class represents individual property data used for training the model. The Model Trainer class uses this data to train and evaluate machine learning models for accurate prediction.

The Flask App class acts as the central controller that connects user input with the machine learning model. It accepts user requests, processes the data using trained models, and returns the predicted property price. This interaction ensures smooth data flow between input, processing, and output, enabling efficient and accurate real-estate price prediction.

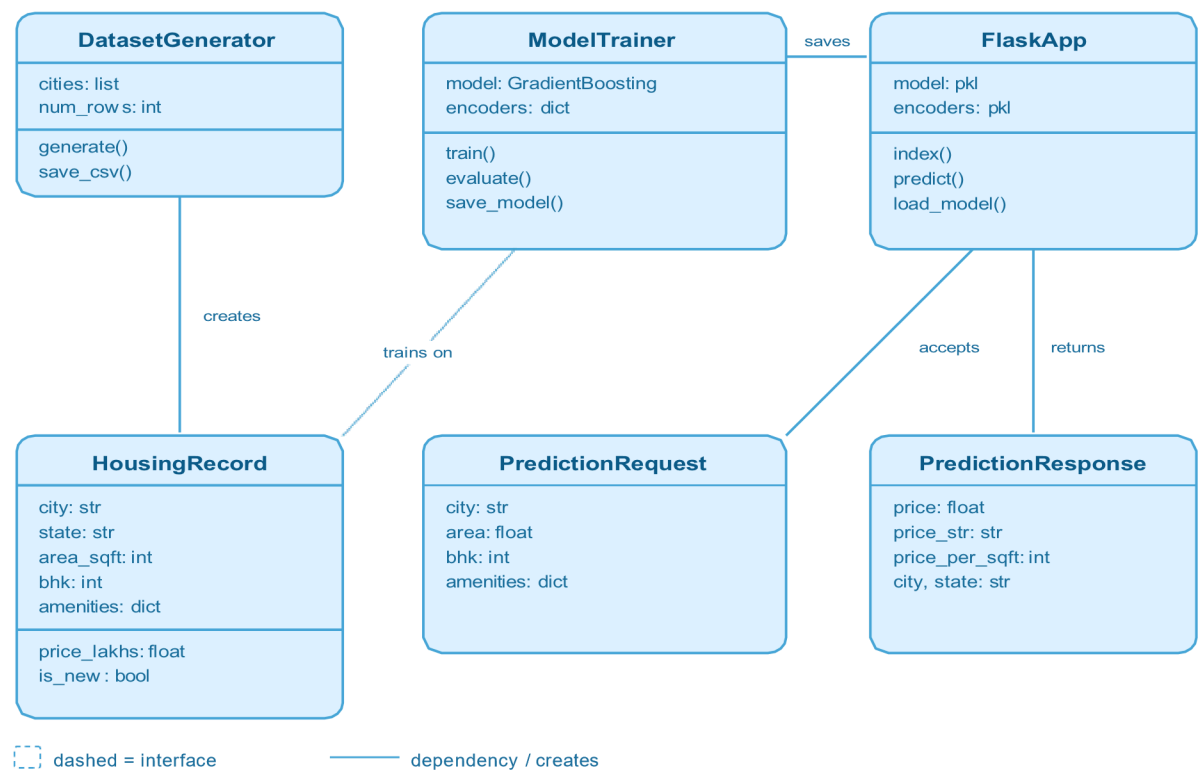


Fig. 4.3.2 Class Diagram

4.3.3 SEQUENCE DIAGRAM

A sequence diagram is a type of UML diagram that illustrates how different components of a system interact with each other in a particular sequence over time. It shows the flow of messages between objects and the order in which operations are performed.

In a sequence diagram, participants (objects or components) are represented by vertical lines called lifelines, and the interactions between them are shown using arrows. These arrows represent messages or function calls exchanged between components.

Sequence diagrams are useful for understanding the dynamic behavior of a system. They help in visualizing how data flows through the system and how different modules coordinate to perform a task.

Sequence Diagram For Real-Estate Price Prediction System

Objects:

- **User** : A user who can enter property details such as location, area, BHK, and amenities, request price prediction, and view the predicted results for better decision-making.
- **System**: The central platform that handles all operations including receiving user input, preprocessing data, applying machine learning models, generating predictions, and displaying results.
- **Admin** : The administrator who manages datasets, updates and trains machine learning models, and monitors overall system performance and accuracy.

Sequence Of Interactions:

User to System:

- The interaction begins when the User accesses the application through a web interface (GET request).
- The System loads the interface (index page) and retrieves necessary metadata such as city and state information from the Dataset.
- The User enters property details such as location, area, BHK, and amenities and sends a prediction request (POST /predict).
- The System receives the input data and performs preprocessing steps such as validation and encoding.
- The processed data is then sent to the ML Model for prediction.
- The ML Model analyzes the input features and generates the predicted property price.
- The System receives the predicted result and formats it into a readable response.
- Finally, the System sends the prediction result back to the User in JSON format, displaying the predicted price and related details.

System to Dataset:

- The System retrieves required data such as city and state information for input validation.
- The Dataset provides training data that has already been used to build the ML Model.

System to ML Model:

- The System sends preprocessed input data to the ML Model.
- The ML Model executes the prediction algorithm using trained parameters.
- ML Model returns the predicted price to the System.

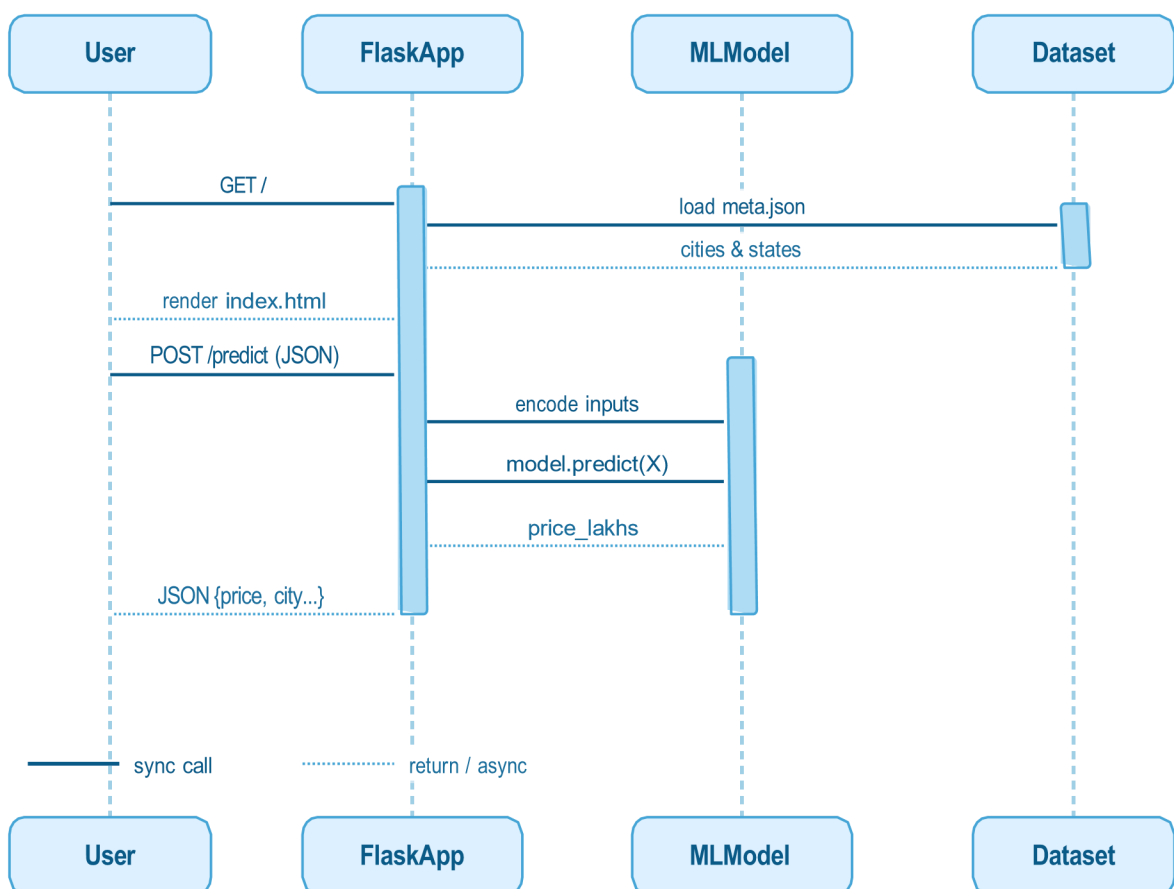


Fig. 4.3.3.1 Sequence diagram

4.3.4 ACTIVITY DIAGRAM

An activity diagram is a type of behavioral diagram in the Unified Modeling Language (UML) that visualizes the flow of activities or actions within a system or process. It is particularly useful for modeling the dynamic aspects of a system, illustrating how different activities are performed and how they interact with each other over time.

Key elements of an activity diagram include:

Activities: Represent tasks or actions performed within the system. Each activity is depicted as a rounded rectangle with a label describing the action.

Actions: Represent specific steps or operations involved in performing an activity. Actions are denoted by rectangles with rounded corners and contain descriptive labels.

Control Flows: Represent the flow of control from one activity or action to another. Control flows are depicted as arrows connecting activities and actions, indicating the sequence in which they occur.

Decision Nodes: Represent decision points within the process flow where the direction of the flow is determined based on certain conditions. Decision nodes are depicted as diamonds and contain labels describing the decision criteria.

Fork and Join Nodes: Represent concurrency in the process flow, where multiple activities can occur simultaneously (fork) or multiple flows converge into a single flow (join).

Swimlanes: Represent partitions or roles responsible for performing certain activities, such as User, System, and Machine Learning Model.

Activity diagrams are commonly used in software development to model system workflows and processes. They provide a clear and concise way to visualize complex operations, identify inefficiencies, and understand how the system functions.

In the **Real-Estate Price Prediction System using Machine Learning**, the activity diagram illustrates the workflow of property price prediction. The process starts when the system receives user input through a prediction request. The system validates the input data, and a decision is made based on whether the input is valid or not. If invalid, an error response is returned; otherwise, the system proceeds with data preprocessing such as encoding categorical values and building feature vectors.

The processed data is then passed to the machine learning model, where prediction is performed using algorithms like Gradient Boosting. The predicted price is formatted into a readable format and finally returned to the user as a response. This workflow ensures efficient and accurate real-estate price prediction.

Activity Diagram For Real-Estate Price Prediction System:

Start: The process begins. User: **1. Access Application**

- o The user opens the web application.

1. The system loads the prediction interface

2. Enter Property Details

- o User inputs details such as:

- Location
- Area (square feet)
- BHK
- Amenities

2. Submit Request

- o The system receives a POST /predict request.
- o The system performs input validation.

3. Decision: Are inputs valid?

- **No:** Display error message → End
- **Yes:** Proceed to data processing

4. Data Processing

- o Encode categorical data (city, state)
- o Build feature vector

5. Prediction Process

- o Input data is sent to the **Machine Learning Model**
- o Model applies algorithm (e.g., Gradient Boosting)
- o System generates predicted price

6. Display Result

- o Format predicted price (Lakhs/Crores)
- o Display result to user

7. End of User Activity

Admin:**1. Login & Authentication**

- o Admin logs into system
- o System validates credentials

2. Manage Dataset

- o Upload or update property dataset
- o System stores updated data

3. Train / Update Model

- o Admin trains machine learning model
- o System evaluates and saves model

4. Monitor System

- o Check system performance and accuracy
- o System displays metrics

5. End of Admin Activity**Control Flow:**

Sequential arrows connect steps from input → processin → prediction → output.

Decision Nodes:

- Input validation check
- Prediction flow (valid / invalid input)

Swimlanes:

- User lane → Handles input and result viewing
- Admin lane → Handles dataset and model management

Conclusion:

This activity diagram captures the complete workflow of the system, including user input, data processing, machine learning prediction, and result generation. It also includes administrative tasks such as dataset management and model training, ensuring efficient and accurate real-estate price prediction.

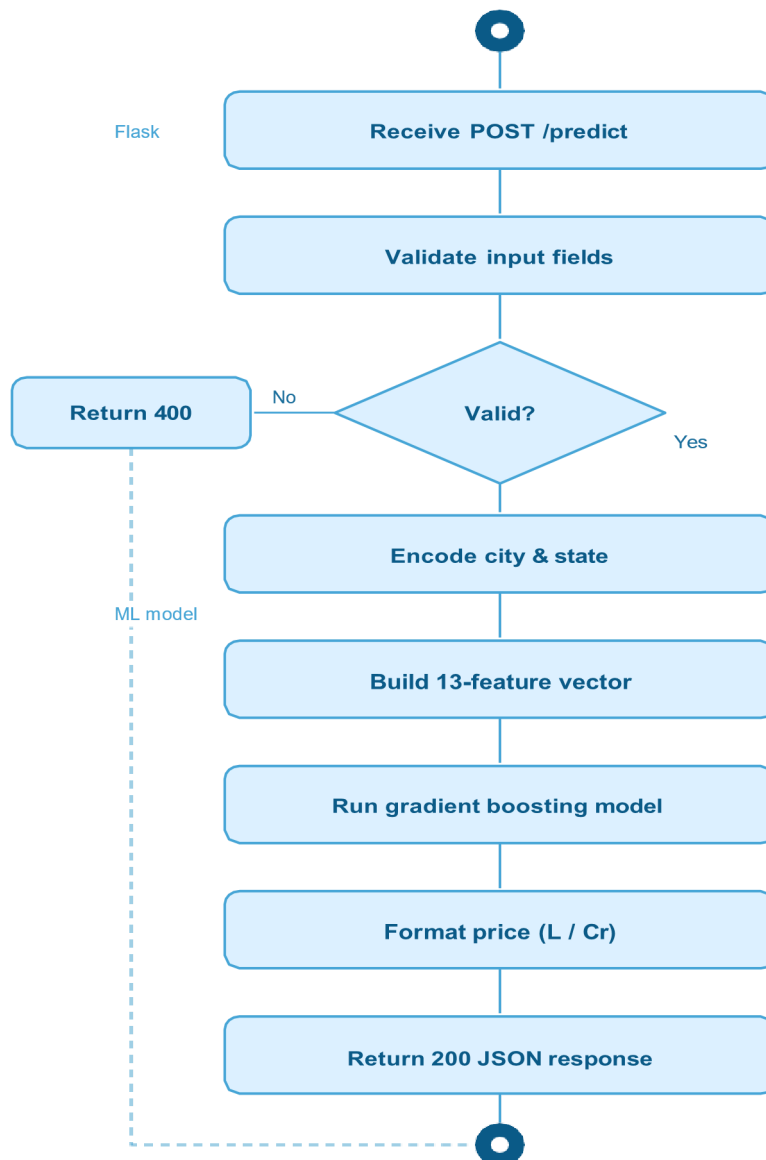


Fig. 4.3.4 Activity Diagram

5.IMPLEMENTATION

5.1 TECHNOLOGY STACK

5.1.1 Frontend Technologies:

HTML Introduction:

HTML (HyperText Markup Language) is used to create the structure of the web application. In the Real-Estate Price Prediction System, HTML is used to design input forms where users enter property details such as location, area, BHK, and amenities. It helps in organizing content like headings, buttons, and result displays.

CSS Introduction:

CSS (Cascading Style Sheets) is used to style the web pages and improve visual appearance. It controls layout, colors, fonts, and responsiveness. In this project, CSS ensures that the application is user-friendly, clean, and accessible across different devices such as desktops and mobiles.

JAVASCRIPT Introduction:

JavaScript is used to make the application interactive. It handles user input, form validation, and communication with the backend using APIs. It enables dynamic updates such as displaying predicted prices without reloading the page.

5.1.2 Backend Technologies:

Python Introduction:

Python is used as the core backend programming language. It is widely used in machine learning due to its simplicity and powerful libraries. In this project, Python handles data processing, model training, and prediction logic.

Flask Introduction:

Flask is a lightweight web framework in Python used to build the backend of the application. It handles user requests, processes input data, connects with the machine learning model, and returns prediction results to the frontend.

Machine Learning Libraries:

Libraries such as **Scikit-learn**, **NumPy**, and **Pandas** are used for data preprocessing, model training, and prediction. These libraries help in handling datasets and implementing algorithms like Linear Regression, Random Forest, and Gradient Boosting.

5.1.3 Database:

Dataset / CSV Files Introduction:

In this project, datasets are used instead of traditional databases. The dataset contains property- related information such as location, area, BHK, amenities, and price. This data is used to train the machine learning model and improve prediction accuracy. The dataset is usually stored in CSV format and processed using Python libraries.

1. Module Description:

The Real-Estate Price Prediction System is divided into several key modules, each responsible for a specific functionality:

1. User Input Module

- Allows users to enter property details such as location, area, BHK, and amenities
- Validates user input before processing
- Sends input data to backend for prediction

2. Data Preprocessing Module

- Handles missing values and cleans data
- Encodes categorical data (location, state)
- Normalizes input data for better model performance

3. Machine Learning Module

- Implements algorithms such as Linear Regression, Random Forest, and Gradient Boosting
- Trains models using dataset
- Evaluates model performance using MAE, MSE, and R^2

4. Prediction Module

- Takes processed input data
- Applies trained model
- Generates predicted property price

5. Result Display Module

- Displays predicted price to the user
- Formats output in readable form (Lakhs/Crores)
- Provides basic insights for decision-making

6. Model Management Module (Admin)

- Allows updating and retraining models
- Manages dataset used for training
- Monitors model performance

7. UI/UX and Responsive Design Module

- Provides simple and user-friendly interface
- Ensures responsive design for all devices
- Displays input forms and results clearly

The Real-Estate Price Prediction System is divided into several key modules, each responsible for a specific functionality:

1. User Input Module

- Allows users to enter property details such as location, area, BHK, and amenities
- Validates user input before processing
- Sends input data to backend for prediction

2. Data Preprocessing Module

- Handles missing values and cleans data
- Encodes categorical data (location, state)
- Normalizes input data for better model performance

3. Machine Learning Module

- Implements algorithms such as Linear Regression, Random Forest, and Gradient Boosting
- Trains models using dataset
- Evaluates model performance using MAE, MSE, and R^2

4. Prediction Module

- Takes processed input data
- Applies trained model
- Generates predicted property price

5. Result Display Module

- Displays predicted price to the user
- Formats output in readable form (Lakhs/Crores)
- Provides basic insights for decision-making

6. Model Management Module (Admin)

- Allows updating and retraining models
- Manages dataset used for training
- Monitors model performance

7. UI/UX and Responsive Design Module

- Provides simple and user-friendly interface
- Ensures responsive design for all devices
- Displays input forms and results clearly

5.2 SAMPLE CODE

5.2.1 Generate_dataset:

```
"""
All-India Real Estate Dataset Generator
200+ locations across all Indian states & UTs
"""

import pandas as
pd import numpy
as np import os
np.random.seed(42)
LOCATIONS =
# Andhra Pradesh
"Vijayawada":("Andhra Pradesh",5200),"Visakhapatnam":("Andhra Pradesh",6800),
"Tirupati":("Andhra Pradesh",5500),"Guntur":("Andhra Pradesh",4800),
"Nellore":("Andhra Pradesh",4200),"Amaravati":("Andhra Pradesh",5000),
"Kakinada":("Andhra Pradesh",4000),"Rajahmundry":("Andhra Pradesh",4500),
# Arunachal Pradesh
"Itanagar":("Arunachal Pradesh",3500),
# Assam
"Guwahati":("Assam",5500),"Dibrugarh":("Assam",3800),
"Silchar":("Assam",3200),"Jorhat":("Assam",3000),
# Bihar
"Patna":("Bihar",5000),"Gaya":("Bihar",3200),
"Muzaffarpur":("Bihar",3000),"Bhagalpur":("Bihar",2800),
# Chhattisgarh
"Raipur":("Chhattisgarh",4500),"Bhilai":("Chhattisgarh",4000),
"Bilaspur":("Chhattisgarh",3500),"Durg":("Chhattisgarh",3200),
# Delhi / NCR
"Connaught Place":("Delhi",22000),"South Delhi":("Delhi",18000),
"Dwarka":("Delhi",10000),"Rohini":("Delhi",8500),
"Lajpat Nagar":("Delhi",14000),"Saket":("Delhi",16000),
"Vasant Kunj":("Delhi",15000),
"Noida Sector 62":("Uttar Pradesh",7500),"Noida Sector 137":("Uttar Pradesh",6800),
"Greater Noida":("Uttar Pradesh",5500),
"Gurgaon DLF Phase":("Haryana",12000),"Gurgaon Sohna Road":("Haryana",9000),
"Faridabad":("Haryana",5500),"Ghaziabad":("Uttar Pradesh",5000),
# Goa
"Panaji":("Goa",9000),"Margao":("Goa",7500),
"Calangute":("Goa",11000),"Mapusa":("Goa",8000),"Vascoda
Gama":("Goa",7000), # Gujarat
"Ahmedabad SG Highway":("Gujarat",7000),"Ahmedabad Bopal":("Gujarat",6000),
"Surat":("Gujarat",5500),"Vadodara":("Gujarat",5200),
"Rajkot":("Gujarat",4500),"Gandhinagar":("Gujarat",5800),
"Anand":("Gujarat",4000),"Bhavnagar":("Gujarat",3800),
# Haryana
"Ambala":("Haryana",4500),"Karnal":("Haryana",4200),
"Hisar":("Haryana",3800),"Panipat":("Haryana",4000),"Sonipat":("Haryana",4500),
# Himachal Pradesh
"Shimla":("Himachal Pradesh",6500),"Manali":("Himachal Pradesh",7000),
"Dharamshala":("Himachal Pradesh",5500),
```

Jharkhand
 "Ranchi":("Jharkhand",4800),"Jamshedpur":("Jharkhand",4500),
 "Dhanbad":("Jharkhand",3500),"Bokaro":("Jharkhand",3200),
 # Karnataka
 "Whitefield Bangalore":("Karnataka",9000),"Koramangala":("Karnataka",14000),
 "Indiranagar Bangalore":("Karnataka",13000),"Electronic City":("Karnataka",7500),
 "Hebbal":("Karnataka",9500),"Sarjapur Road":("Karnataka",8500),
 "JP Nagar":("Karnataka",9000),"Mysuru":("Karnataka",5500),
 "Mangaluru":("Karnataka",6000),"Hubballi":("Karnataka",4500),
 "Belagavi":("Karnataka",4200),"Tumkur":("Karnataka",4000), # Kerala
 "Thiruvananthapuram":("Kerala",6500),"Kochi":("Kerala",8000),
 "Kozhikode":("Kerala",6000),"Thrissur":("Kerala",6200),
 "Kollam":("Kerala",5500),"Kottayam":("Kerala",5800),
 # Madhya Pradesh
 "Bhopal":("Madhya Pradesh",5000),"Indore":("Madhya Pradesh",6000),
 "Jabalpur":("Madhya Pradesh",4000),"Gwalior":("Madhya Pradesh",4200),
 "Ujjain":("Madhya Pradesh",3800),
 # Maharashtra
 "Bandra West":("Maharashtra",35000),"Lower Parel":("Maharashtra",30000),
 "Worli":("Maharashtra",45000),"Andheri West":("Maharashtra",22000),
 "Powai":("Maharashtra",20000),"Thane West":("Maharashtra",11000),
 "Navi Mumbai":("Maharashtra",9500),
 "Pune Koregaon Park":("Maharashtra",12000),"Pune Hadapsar":("Maharashtra",7500),
 "Pune Hinjewadi":("Maharashtra",8000),"Pune Wakad":("Maharashtra",8500),
 "Nashik":("Maharashtra",5500),"Nagpur":("Maharashtra",5800),
 "Aurangabad":("Maharashtra",4800),"Solapur":("Maharashtra",4000),
 "Kolhapur":("Maharashtra",5000),
 # Manipur
 "Imphal":("Manipur",3500),
 # Meghalaya
 "Shillong":("Meghalaya",5000),
 # Mizoram
 "Aizawl":("Mizoram",3800),
 # Nagaland
 "Kohima":("Nagaland",3500),"Dimapur":("Nagaland",4000),
 # Odisha
 "Bhubaneswar":("Odisha",5500),"Cuttack":("Odisha",4500),
 "Rourkela":("Odisha",3800),"Berhampur":("Odisha",3500),
 # Punjab
 "Amritsar":("Punjab",5000),"Ludhiana":("Punjab",5500),
 "Jalandhar":("Punjab",4800),"Mohali":("Punjab",6500),
 "Patiala":("Punjab",4200),"Chandigarh":("Punjab",8000),
 # Rajasthan
 "Jaipur Vaishali Nagar":("Rajasthan",6500),"Jaipur Malviya Nagar":("Rajasthan",7000),
 "Jodhpur":("Rajasthan",4800),"Udaipur":("Rajasthan",5500),
 "Kota":("Rajasthan",4200),"Ajmer":("Rajasthan",4000),
 # Sikkim
 "Gangtok":("Sikkim",5500),
 # Tamil Nadu
 "Anna Nagar Chennai":("Tamil Nadu",11000),"OMR Chennai":("Tamil Nadu",8500),
 "Adyar":("Tamil Nadu",13000),"Velachery":("Tamil Nadu",9000),


```

"Perambur":("Tamil Nadu",7000),"Coimbatore":("Tamil Nadu",5500),
"Madurai":("Tamil Nadu",5000),"Tiruchirappalli":("Tamil Nadu",4800),
"Salem":("Tamil Nadu",4200),"Tirunelveli":("Tamil Nadu",4000),
# Telangana
"Banjara Hills":("Telangana",16000),"Jubilee Hills":("Telangana",18000),
"Gachibowli":("Telangana",10000),"Kondapur":("Telangana",9500),
"Madhapur":("Telangana",10500),"Kukatpally":("Telangana",8000),
"LB Nagar":("Telangana",7000),"Warangal":("Telangana",4500),
# Tripura
"Agartala":("Tripura",3800),
# Uttar Pradesh
"Lucknow Gomti Nagar":("Uttar Pradesh",6000),"Lucknow Hazratganj":("Uttar Pradesh",6500),
"Agra":("Uttar Pradesh",4500),"Varanasi":("Uttar Pradesh",4800),
"Kanpur":("Uttar Pradesh",4000),"Allahabad":("Uttar Pradesh",4200),
"Meerut":("Uttar Pradesh",4500),
# Uttarakhand
"Dehradun":("Uttarakhand",6000),"Haridwar":("Uttarakhand",5000),
"Nainital":("Uttarakhand",6500),
# West Bengal
"Salt Lake Kolkata":("West Bengal",8000),"New Town Kolkata":("West Bengal",7000),
"South Kolkata":("West Bengal",7500),"Howrah":("West Bengal",5500),
"Siliguri":("West Bengal",4500),"Durgapur":("West Bengal",4000),
"Asansol":("West Bengal",3500),
# Union Territories
"Puducherry":("Puducherry",5500),"Port Blair":("Andaman & Nicobar",6000),
"Jammu":("J&K",5500),"Srinagar":("J&K",6000),"Leh":("Ladakh",7000),
}

```

N = 10000

```

def generate_dataset(n=N):
    city_names = list(LOCATIONS.keys()) cities =
    np.random.choice(city_names, n) states =
    np.array([LOCATIONS[c][0] for c in cities]) base_px
    = np.array([LOCATIONS[c][1] for c in cities])

    area = np.random.randint(350, 5001, n)
    bhk = np.random.choice([1,2,3,4,5], n, p=[0.12,0.33,0.32,0.16,0.07])
    area = np.maximum(area, bhk*280)

    gymnasium = np.random.choice([0,1], n, p=[0.42,0.58])
    indoor_games = np.random.choice([0,1], n, p=[0.52,0.48])
    car_parking = np.random.choice([0,1], n, p=[0.28,0.72])
    jogging_track = np.random.choice([0,1], n, p=[0.56,0.44])
    swimming_pool = np.random.choice([0,1], n, p=[0.65,0.35])
    security = np.random.choice([0,1], n, p=[0.20,0.80])
    is_new = np.random.choice([0,1], n, p=[0.44,0.56])

    total_floors = np.random.choice([3,5,7,10,15,20,25,30], n,
    p=[0.10,0.15,0.20,0.20,0.15,0.10,0.06,0.04])
    floor_no = np.array([np.random.randint(0, tf+1) for tf in total_floors])

```

```

price = (base_px * area / 100000).astype(float)
price *= (1 + (bhk-1)*0.04)
price += gymnasium * np.random.uniform(1.5, 6, n) price +=
indoor_games * np.random.uniform(1.0, 4, n) price += car_parking
* np.random.uniform(2.0, 8, n) price += jogging_track *
np.random.uniform(0.8, 3, n) price += swimming_pool *
np.random.uniform(3.0, 10, n) price += security *
np.random.uniform(0.5, 2.5, n) price *= np.where(is_new==1,
np.random.uniform(1.04,1.14,n), 1.0) price *=
np.where(floor_no>5, 1+(floor_no-5)*0.005, 1.0) price *=
np.random.uniform(0.94, 1.06, n) price = np.round(price, 2)

return pd.DataFrame({
    "city":city_names[0] if False else cities,
    "state":states,"area_sqft":area,"bhk":bhk,
    "floor_no":floor_no,"total_floors":total_floors,
    "gymnasium":gymnasium,"indoor_games":indoor_games,
    "car_parking":car_parking,"jogging_track":jogging_track,
    "swimming_pool":swimming_pool,"security":security,
    "is_new":is_new,"price_lakhs":price,
})

if __name__ == "__main__": df =
    generate_dataset() os.makedirs("data",
    exist_ok=True)
    df.to_csv("data/india_housing.csv",
    index=False)
    print(f"Dataset: {len(df)} rows | {df['city'].unique()} cities | {df['state'].unique()} states")
    print(df.head(3))

```

5.2.2 Train_model:

```

"""
All-India Real Estate Price Prediction - Model Training
"""

import pandas as pd import numpy as np import os,
json, pickle from sklearn.model_selection import
train_test_split from sklearn.preprocessing import
LabelEncoder
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("data/india_housing.csv")
print(f"Loaded {len(df)} rows | {df['city'].unique()} cities | {df['state'].unique()} states\n")

le_city=LabelEncoder() le_state = LabelEncoder()
df["city_enc"] = le_city.fit_transform(df["city"])
df["state_enc"]=
le_state.fit_transform(df["state"])

FEATURES = ["city_enc","state_enc","area_sqft","bhk","floor_no","total_floors",

```

```

        "gymnasium","indoor_games","car_parking","jogging_track",
        "swimming_pool","security","is_new"]
TARGET = "price_lakhs"

X, y = df[FEATURES], df[TARGET]

X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.10, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.111, random_state=42)

print(f"Train: {len(X_train)} Val: {len(X_val)} Test: {len(X_test)}\n")

os.makedirs("data", exist_ok=True)
for split, Xs, ys in [
    ("train",X_train,y_train),("val",X_val,y_val),("test",X_test,y_test)]:
    d = Xs.copy();
    d[TARGET]=ys; d.to_csv(f"data/{split}.csv", index=False)

models = {
    "LinearRegression": LinearRegression(),
    "RandomForest": RandomForestRegressor(n_estimators=150, random_state=42, n_jobs=-1),
    "GradientBoosting": GradientBoostingRegressor(n_estimators=300, learning_rate=0.08,
        max_depth=6, random_state=42),
    }

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    r2 = r2_score(y_test, pred)
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    results[name]={"R2":round(r2,4),"MAE":round(mae,2),"RMSE":round(rmse,2)}
    print(f"{name:25s} R²={r2:.4f} MAE={mae:.2f} RMSE={rmse:.2f}")

os.makedirs("model", exist_ok=True)
best = models["GradientBoosting"]
pickle.dump(best, open("model/model.pkl","wb"))
pickle.dump(le_city, open("model/le_city.pkl","wb"))
pickle.dump(le_state, open("model/le_state.pkl","wb"))

# city -> state mapping for frontend
city_state = {c: LOCS[c][0] if False else df.groupby("city")["state"].first().to_dict()[c]
               for c in le_city.classes_}

meta = {
    "cities": sorted(le_city.classes_.tolist()),
    "states": sorted(le_state.classes_.tolist()),
    "city_state": {c: df[df.city==c]["state"].iloc[0] for c in le_city.classes_},
}
json.dump(meta, open("model/meta.json","w"), indent=2)
json.dump(results, open("model/metrics.json","w"), indent=2)
print(f"\n Saved — {len(meta['cities'])} cities, {len(meta['states'])} states")

```

5.2.3 App

```
"""All-India Real Estate Price Prediction - Flask App"""
from flask import Flask, render_template, request, jsonify
import pickle, json, numpy as np, os

app = Flask(__name__)
BASE = os.path.dirname(os.path.abspath(__file__))

model = pickle.load(open(os.path.join(BASE,"model/model.pkl"),"rb"))
le_city = pickle.load(open(os.path.join(BASE,"model/le_city.pkl"),"rb"))
le_state = pickle.load(open(os.path.join(BASE,"model/le_state.pkl"),"rb"))
meta = json.load(open(os.path.join(BASE,"model/meta.json")))
metrics = json.load(open(os.path.join(BASE,"model/metrics.json")))

@app.route("/")
def index():
    return render_template("index.html", meta=meta, metrics=metrics)

@app.route("/predict", methods=["POST"])
def predict():
    try:
        d = request.get_json()
        city = d.get("city","")
        area= float(d.get("area",0))
        bhk = int(d.get("bhk",1))
        floor_no= int(d.get("floor_no",0))
        total_floors = int(d.get("total_floors",5))
        gymnasium = int(d.get("gymnasium",0))
        indoor_games = int(d.get("indoor_games",0))
        car_parking = int(d.get("car_parking",0))
        jogging_track = int(d.get("jogging_track",0))
        swimming_pool =
        int(d.get("swimming_pool",0)) security      =
        int(d.get("security",0)) is_new           =
        int(d.get("is_new",0))

        if not city or area<=0 or bhk<=0:
            return jsonify({"error":"Fill Location, Area and BHK."}), 400
        if city not in le_city.classes_:
            return jsonify({"error":f"Unknown city: {city}"}), 400

        state= meta["city_state"][city]
        city_enc = le_city.transform([city])[0]
        state_enc=le_state.transform([state])[0]

        feat = np.array([[city_enc, state_enc, area, bhk, floor_no,
            total_floors, gymnasium, indoor_games, car_parking, jogging_track,
```

```

        swimming_pool, security, is_new]]) price =
        float(model.predict(feat)[0])

        if price >= 100:
            price_str = f"₹ {price/100:.2f} Cr"
        else:
            price_str = f"₹ {price:.2f} Lakhs"

        return jsonify({
            "price": round(price,2), "price_str": price_str,
            "price_per_sqft": round((price*100000)/area),
            "city": city, "state": state, "area": area, "bhk": bhk,
        })
    except
Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)

```

5.2.4 Signup.jsx

```

import React, { useContext, useState } from "react";
import AuthLayout from "../../components/Layouts/AuthLayout";
import { Link, useNavigate } from "react-router-dom"; import
Input from "../../components/Inputs/Input"; import {
validateEmail } from "../../utils/helper";
import ProfilePhotoSelector from "../../components/Inputs/ProfilePhotoSelector";
import { API_PATHS } from "../../utils/apiPaths"; import uploadImage from
 "../../utils/uploadImage"; import { UserContext } from
 "../../context/UserContext"; import axiosInstance from "../../utils/axiosInstance";

const SignUpForm = () => {
    const [profilePic, setProfilePic] = useState(null);
    const [fullName, setFullName] = useState("");
    const [email, setEmail] = useState(""); const
    [password, setPassword] = useState(""); const
    [error, setError] = useState(null);

    const { updateUser } = useContext(UserContext);
    const navigate = useNavigate();

    // Handle Sign Up
    Form Submit const handleSignUp =
    async (e) => { e.preventDefault();
    let profileImageUrl = "";

    if (!fullName) {
        setError("Please enter your name");
        return;
    }

```

```

    }

    if (!validateEmail(email)) {
      setError("Please enter a valid email
address."); return;
    }

    if (!password) {
      setError("Please enter the password");
      return;
    }

setError("");

//
SignUp API Call try
{
  // Upload image if present
    if (profilePic) {
      const imgUploadRes = await uploadImage(profilePic);
      profileImageUrl = imgUploadRes.imageUrl || ""; }

  const response = await axiosInstance.post(API_PATHS.AUTH.REGISTER, {
    fullName,
    email,
    password,
    profileImageUrl,
  });

const { token, user } = response.data;
if (token) {
  localStorage.setItem("token", token);
  updateUser(user);
  navigate("/dashboard");
}
} catch (error) {
  if (error.response &&
error.response.data.message) {
    setError(error.response.data.message);
  } else {
    setError("Something went
wrong. Please try again.");
  }
}

return (
<AuthLayout>
<div className="lg:w-[100%] h-auto md:h-full mt-10 md:mt-0 flex flex-col justify-center">
  <h3 className="text-xl font-semibold text-black">Create an Account</h3>
  <p className="text-xs text-slate-700 mt-[5px]
mb-6"> Join us today by entering your details below.
</p>

  <form onSubmit={handleSignUp}>

```

```

<ProfilePhotoSelector image={profilePic} setImage={setProfilePic} />

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  <Input
    value={fullName}
    onChange={({target }) => setFullName(target.value)}
    label="Full Name" placeholder="Your Name"
    type="text"
  />

  <Input
    value={email}
    onChange={({target }) =>
      setEmail(target.value)} label="Email Address"
    placeholder="Your email"
    type="text"
  />

  <div className="col-span-2">
    <Input
      value={password}
      onChange={({target }) =>
        setPassword(target.value)} label="Password"
      placeholder="Min 8 Characters" type="password"
    />
  </div>
</div>

{error && <p className="text-red-500 text-xs pb-2.5">{error}</p>}}

<button type="submit" className="btn-primary">
  SIGN UP
</button>

<p className="text-[13px] text-slate-800 mt-3">
  Already have an account?{" "}
<Link className="font-medium text-primary underline" to="/login">
  Login
</Link>
</p>
</form>
</div>
</AuthLayout>
);
};

export default SignUpForm;

```

5.2.5 Index

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width,initial-scale=1.0"/>
<title>India House Pricing</title>
<style>
*{margin:0;padding:0;box-sizing:border-box;}
body{
  min-height:100vh;
background:linear-gradient(135deg,#6a11cb 0%,#2575fc 35%,#00c9ff 65%,#92fe9d100%);
display:flex;flex-direction:column;align-items:center;justify-content:center; font-family:'Segoe
UI',Tahoma,Geneva,Verdana,sans-serif;padding:24px 12px;
}
.hero{text-align:center;margin-bottom:22px;color:#fff;}
.hero h1{font-size:2.2rem;font-weight:800;text-shadow:0 2px 12px rgba(0,0,0,.2);}
.hero p{font-size:1rem;margin-top:6px;opacity:.88;}
.card{background:#fff;border-radius:18px;padding:28px 32px 24px;width:680px;max-width:96vw;
  box-shadow:0 20px 60px rgba(0,0,0,.18);}
.row{display:flex;gap:18px;margin-bottom:16px;flex-wrap:wrap;}
.field{flex:1;min-width:200px;display:flex;flex-direction:column;}
.fieldlabel{font-size:.75rem;font-weight:700;color:#999;text-transform:uppercase;
  letter-spacing:.8px;margin-bottom:5px;}
.field select,.field input{border:none;border-bottom:2px solid #e0e0e0;padding:8px 4px;
  font-size:.93rem;outline:none;color:#333;background:transparent;transition:border-color .2s;}
.field select:focus,.field input:focus{border-bottom-color:#2575fc;}
.divider{border:none;border-top:1px solid #f0f0f0;margin:12px 0;}
.section-title{font-size:.72rem;font-weight:700;color:#aaa;text-transform:uppercase;
  letter-spacing:1px;margin-bottom:10px;}
.toggles-grid{display:grid;grid-template-columns:1fr 1fr 1fr;gap:10px 14px;margin-bottom:14px;}
.toggle-item{display:flex;align-items:center;gap:8px;font-size:.86rem;color:#444;font-weight:500;}
.switch{position:relative;display:inline-block;width:42px;height:22px;}
.switch input{opacity:0;width:0;height:0;}
.slider{position:absolute;cursor:pointer;top:0;left:0;right:0;bottom:0;
  background:#ccc;border-radius:22px;transition:.3s;}
.slider:before{position:absolute;content:"";height:16px;width:16px;left:3px;bottom:3px;
  background:#fff;border-radius:50%;transition:.3s;}
input:checked+.slider{background:#2575fc;}
input:checked+.slider:before{transform:translateX(20px);}
.new-resale{display:flex;align-items:center;gap:10px;font-size:.9rem;color:#555;font-weight:600;}
.btn-predict{display:block;margin:18px auto 0;padding:12px 52px;background:#2575fc;
color:#fff;border:none;border-radius:8px;font-size:1rem;font-weight:700;cursor:pointer;
box-shadow:0 4px 16px rgba(37,117,252,.35);transition:background .2s,transform .1s;}
.btn-predict:hover{background:#1a5fd4;transform:translateY(-1px);}
#result-box{display:none;margin-top:20px;padding:18px 20px;
  background:linear-gradient(135deg,#e0f0ff,#f0e8ff);border-radius:12px;text-align:center;}
#result-box .price-main{font-size:2rem;font-weight:800;color:#2575fc;}
#result-box .price-sub{font-size:.86rem;color:#666;margin-top:4px;}
#result-box .details{display:flex;justify-content:center;gap:24px;margin-top:12px;flex-wrap:wrap;}
#result-box .detail-item{font-size:.8rem;color:#666;}
#result-box .detail-item strong{display:block;font-size:.95rem;color:#333;}
#error-msg{display:none;margin-top:12px;padding:10px 14px;background:#fff0f0;
  border-left:4px solid #e74c3c;border-radius:6px;color:#c0392b;font-size:.88rem;}
.metrics{margin-top:14px;font-size:.73rem;color:rgba(255,255,255,.78);text-align:center;}
.loader{display:none;width:18px;height:18px;border:3px solid #fff;border-top-color:transparent;
  border-radius:50%;animation:spin .7s linear infinite;margin:0 auto;}
```



```

@keyframes spin{to{transform:rotate(360deg)}}
</style>
</head>
<body>
<div class="hero">
  <h1>Real Estate Price Prediction</h1>
  <p>Covering all of India — 162 cities across 33 states & UTs</p>
</div>

<div class="card">
  <!-- Row 1: State + City -->
  <div class="row">
    <div class="field">
      <label>State / UT</label>
      <select id="state" onchange="filterCities()">
        <option value="">All States</option>
        {% for s in meta.states|sort %}
        <option value="{{ s }}">{{ s }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label>City / Location</label>
      <select id="city">
        <option value="">Select City</option>
        {% for c in meta.cities|sort %}
        <option value="{{ c }}" data-state="{{ meta.city_state[c] }}">{{ c }}</option>
        {% endfor %}
      </select>
    </div>
  </div>

  <!-- Row 2: Area + BHK -->
  <div class="row">
    <div class="field">
      <label>Area (Sq/Ft)</label>
      <input type="number" id="area" placeholder="Enter area in Sq/Ft" min="100"/>
    </div>
    <div class="field">
      <label>No of BHK</label>
      <input type="number" id="bhk" placeholder="e.g. 2" min="1" max="10"/>
    </div>
  </div>

  <!-- Row 3: Floor info + New/Resale -->
  <div class="row">
    <div class="field">
      <label>Floor Number</label>
      <input type="number" id="floor-no" placeholder="e.g. 5" min="0"/>
    </div>
    <div class="field">
      <label>Total Floors</label>
      <input type="number" id="total-floors" placeholder="e.g. 15" min="1"/>
    </div>
    <div class="field" style="flex-direction: row; align-items: center; gap: 12px; padding-bottom: 8px; min-width: auto;">
      <span class="new-resale">New</span>

```



```

<script>
const cityState = {{ meta.city_state | tojson }};
function filterCities() {
    const sel = document.getElementById('state').value;
    const cityDd = document.getElementById('city');
    Array.from(cityDd.options).forEach(o => {
        if (!o.value) { o.style.display = ""; return; }
        o.style.display = (!sel || o.dataset.state === sel) ? "" :
        'none'; }); cityDd.value = "";
    }

    async function predict() {
        const city = document.getElementById('city').value;
        const area = document.getElementById('area').value; const bhk =
        document.getElementById('bhk').value; const floorNo =
        parseInt(document.getElementById('floor-no').value) || 0; const totalFloors =
        parseInt(document.getElementById('total-floors').value) || 5; const isNew =
        document.getElementById('is-new').checked ? 0 : 1; const gym =
        document.getElementById('gymnasium').checked ? 1 : 0; const ig =
        document.getElementById('indoor-games').checked ? 1 : 0; const cp =
        document.getElementById('car-parking').checked ? 1 : 0; const jt =
        document.getElementById('jogging-track').checked ? 1 : 0; const sp =
        document.getElementById('swimming-pool').checked ? 1 : 0; const sec =
        document.getElementById('security').checked ? 1 : 0;

        document.getElementById('result-box').style.display = 'none';
        document.getElementById('error-msg').style.display = 'none';
        if (!city || !area || !bhk) { showError('Please select City and enter Area & BHK. '); return; }

        document.getElementById('btn-text').style.display = 'none';
        document.getElementById('loader').style.display = 'block';

        try {
            const res = await fetch('/predict', {
                method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
                JSON.stringify({ city, area: parseFloat(area), bhk: parseInt(bhk),
                floor_no: floorNo, total_floors: totalFloors,
                gymnasium: gym, indoor_games: ig, car_parking: cp,
                jogging_track: jt, swimming_pool: sp, security: sec, is_new: isNew })
            });
            const data = await res.json();
            if (data.error) { showError(data.error); }
            else {
                document.getElementById('price-display').textContent = data.price_str;
                document.getElementById('price-sub').textContent =
                `Estimated price for ${data.bhk} BHK in ${data.city},
                ${data.state}`; document.getElementById('d-city').textContent =
                data.city; document.getElementById('d-state').textContent = data.state;
                document.getElementById('d-area').textContent = Number(data.area).toLocaleString();
                document.getElementById('d-bhk').textContent = data.bhk + ' BHK';
                document.getElementById('d-ppsf').textContent = '₹ ' + data.price_per_sqft.toLocaleString();
                document.getElementById('result-box').style.display = 'block'; }
            } catch (e) { showError('Server error. Please try again. '); }
            finally {
                document.getElementById('btn-text').style.display = 'inline'
                ; document.getElementById('loader').style.display = 'none';
            }
        }
    }
}

```

```

function showError(msg){
    const el=document.getElementById('error-msg'); el.textContent=msg;
    el.style.display='block';
}
</script>
</body>
</html>

```

6.TESTING

6.1 INTRODUCTION

Testing plays a crucial role in ensuring the quality, reliability, and functionality of the **Real-Estate Price Prediction System using Machine Learning**. It helps in identifying errors, validating system requirements, and improving the overall performance of the application. A properly tested system ensures that all features work accurately and efficiently, providing reliable property price predictions to users.

This chapter outlines the various types of testing carried out to ensure the system functions correctly under all expected conditions.

6.1 Types of Testing

1. Functional Testing

Verifies that each function of the Real-Estate Price Prediction System operates as expected.

Major features tested include:

- **User Input Handling:** Validating property details such as location, area, BHK, and amenities.
- **Data Preprocessing:** Ensuring correct handling of missing values, encoding, and normalization.
- **Prediction Module:** Testing machine learning models for accurate price prediction.
- **Result Display:** Ensuring predicted property prices are correctly displayed.
- **Multiple Predictions:** Verifying repeated predictions with different inputs.

2. User Interface Testing

Ensures that all UI elements are:

- **Intuitive and Accessible:** Easy input forms and clear output display.
- **Cross-Browser Compatible:** Verified on Chrome, Firefox, and Edge.
- **Device Responsive:** Works well on desktops, tablets, and smartphones.

3. Integration Testing

Tests the interactions between different modules to ensure smooth data flow:

- **Frontend & Backend Integration:** Ensures correct data flow between user interface and backend system.
- **Data Processing Integration:** Verifies preprocessing before prediction.
- **Model Integration:** Ensures ML model works properly with backend.
- **End-to-End Testing:** Input → Processing → Prediction → Output flow works correctly.

4. Performance Testing

Evaluates the system's performance under expected conditions:

- **Response Time:** Fast generation of prediction results.
- **Processing Efficiency:** Efficient handling of data and model execution.
- **Scalability:** System supports increasing number of users and predictions.
- **Stress Testing:** Handles multiple prediction requests without failure.

5. Security Testing

Identifies and prevents vulnerabilities to ensure system safety:

- **Input Validation:** Prevents invalid or harmful data inputs.
- **Data Protection:** Ensures safe handling of user input and results.
- **Access Control:** Restricts admin functionalities to authorized users.
- **System Integrity:** Protects model and dataset from unauthorized changes.

7. DEPLOYMENT

The deployment of the **Real-Estate Price Prediction System using Machine Learning** involves hosting the web application so that users can access it over the internet. This section describes the tools, environment, and steps required to deploy the system efficiently and ensure smooth operation.

Deployment Environment

- **Frontend Technologies:** HTML, CSS, JavaScript (or simple UI framework)
- **Backend:** Python (Flask / Django)
- **Machine Learning Libraries:** Scikit-learn, NumPy, Pandas
- **Dataset Storage:** CSV files or database (optional)
- **Web Server:** Flask server or cloud hosting services
- **Operating System:** Windows (development), Linux server (deployment)

- **Hosting Platform:**
 - Frontend: Vercel / Netlify
 - Backend: Render / Railway
 - Dataset/Model: Stored in server or cloud storage

7.2 Deployment Steps

Local Deployment (Development Environment):

1. Install Python and required libraries (NumPy, Pandas, Scikit-learn, Flask).
2. Clone the project repository to your local system.
3. Install dependencies using pip: `pip install -r requirements.txt`
4. Place the dataset (CSV file) in the project directory.

5. Train the machine learning model or load the saved model file.
6. Run the backend server: `python app.py`
7. Open the application in browser: Frontend: `http://localhost:5000`
API Endpoint: `http://localhost:5000/predict`

7.3 Live Server Deployment

Backend Deployment (Render / Railway):

1. Create an account on Render or Railway.
2. Upload your backend project to GitHub.
3. Connect the repository to the hosting platform.
4. Set environment variables (if required).
5. Deploy the backend; you will get a URL like:
`https://your-backend.onrender.com`

Dataset / Model Deployment:

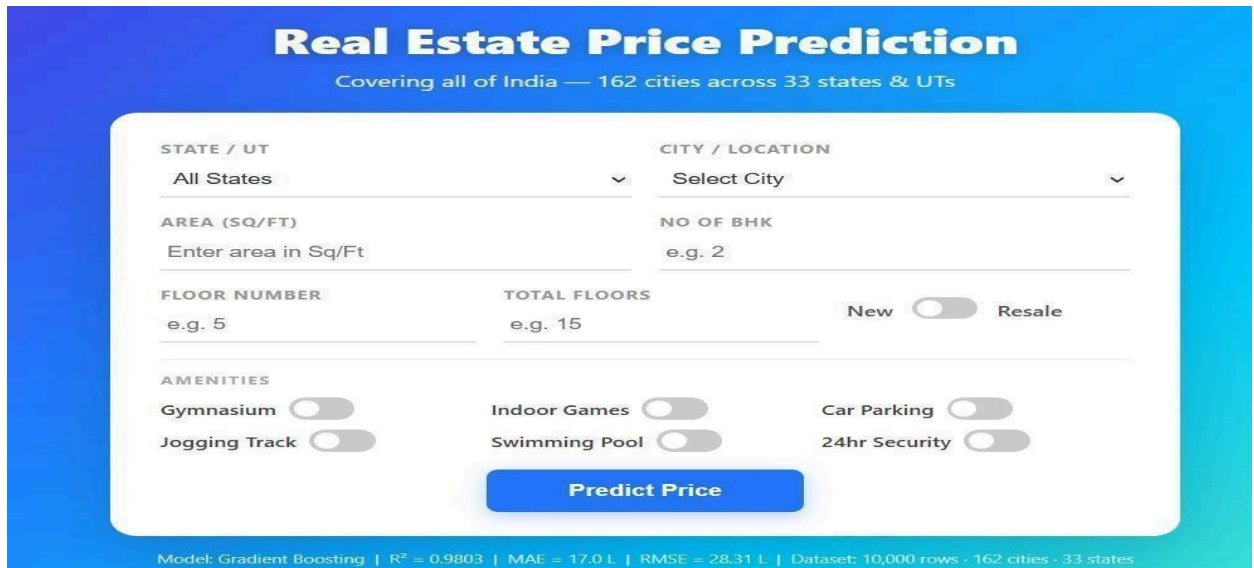
1. Upload trained model file (.pkl) to the server.
2. Ensure dataset is available or integrated into the system.
3. Configure model loading in backend code.

Frontend Deployment (Vercel / Netlify):

1. Upload frontend code to GitHub.
2. Connect repository to Vercel or Netlify.
3. Deploy the application.
4. Configure API URL to connect with backend.
5. Access the application using provided URL.

8.OUTPUT SCREEN

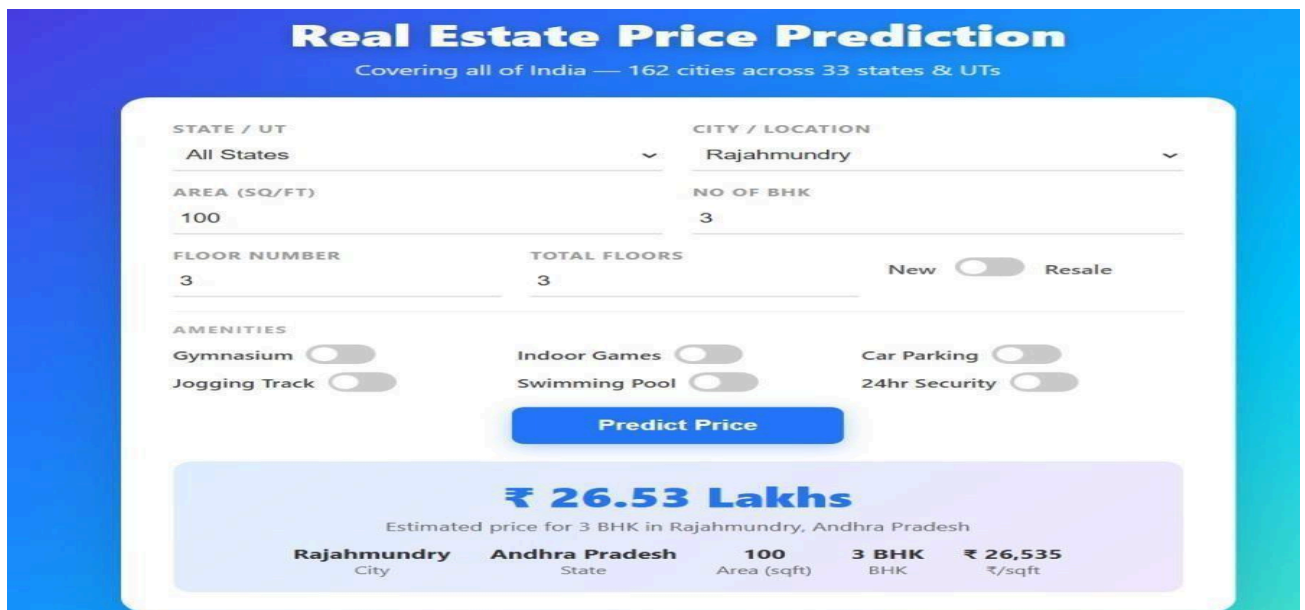
1. Home page:



The screenshot shows the home page of the 'Real Estate Price Prediction' application. The title is 'Real Estate Price Prediction' with a subtitle 'Covering all of India — 162 cities across 33 states & UTs'. The form includes fields for 'STATE / UT' (set to 'All States'), 'CITY / LOCATION' (set to 'Select City'), 'AREA (SQ/FT)' (placeholder 'Enter area in Sq/Ft'), 'NO OF BHK' (placeholder 'e.g. 2'), 'FLOOR NUMBER' (placeholder 'e.g. 5'), and 'TOTAL FLOORS' (placeholder 'e.g. 15'). There are also toggle switches for 'New' and 'Resale' properties, and a section for 'AMENITIES' including 'Gymnasium', 'Indoor Games', 'Car Parking', 'Jogging Track', 'Swimming Pool', and '24hr Security'. A blue 'Predict Price' button is at the bottom. At the very bottom, model statistics are displayed: 'Model: Gradient Boosting | $R^2 = 0.9803$ | MAE = 17.0 L | RMSE = 28.31 L | Dataset: 10,000 rows · 162 cities · 33 states'.

Fig. 8.1 Login Page for the Real-Estate Price Prediction System using Machine Learning

2. Prediction Result:



The screenshot shows the prediction result page of the 'Real Estate Price Prediction' application. The title is 'Real Estate Price Prediction' with a subtitle 'Covering all of India — 162 cities across 33 states & UTs'. The form fields are filled with: 'STATE / UT' (All States), 'CITY / LOCATION' (Rajahmundry), 'AREA (SQ/FT)' (100), 'NO OF BHK' (3), 'FLOOR NUMBER' (3), and 'TOTAL FLOORS' (3). The 'New' toggle is selected. The 'AMENITIES' section shows all toggles (Gymnasium, Indoor Games, Car Parking, Jogging Track, Swimming Pool, 24hr Security) are turned off. A blue 'Predict Price' button is present. Below the button, the predicted price is displayed as '₹ 26.53 Lakhs'. Below this, a summary table provides details: 'Estimated price for 3 BHK in Rajahmundry, Andhra Pradesh'. The table has five columns: City, State, Area (sqft), BHK, and Price (₹/sqft). The values are: Rajahmundry, Andhra Pradesh, 100, 3 BHK, and ₹ 26,535.

City	State	Area (sqft)	BHK	Price (₹/sqft)
Rajahmundry	Andhra Pradesh	100	3 BHK	₹ 26,535

Fig. 8.3 Prediction result for the .Real-Estate Price Prediction System using Machine Learning

3. Prediction with Amenities Enabled::

Real Estate Price Prediction
Covering all of India — 162 cities across 33 states & UTs

STATE / UT: Andhra Pradesh
CITY / LOCATION: Rajahmundry

AREA (SQ/FT): 100
NO OF BHK: 4

FLOOR NUMBER: 3
TOTAL FLOORS: 3
New ☒ Resale ☐

AMENITIES:
Gymnasium ☐ Indoor Games ☒ Car Parking ☐
Jogging Track ☒ Swimming Pool ☒ 24hr Security ☒

Predict Price

₹ 34.02 Lakhs
Estimated price for 4 BHK in Rajahmundry, Andhra Pradesh

Rajahmundry City	Andhra Pradesh State	100 Area (sqft)	4 BHK BHK	₹ 34,023 ₹/sqft
----------------------------	--------------------------------	---------------------------	---------------------	---------------------------

Model: Gradient Boosting | $R^2 = 0.9803$ | MAE = 17.0 L | RMSE = 28.31 L | Dataset: 10,000 rows - 162 cities - 33 states

Fig. 8.4 Prediction with Amenities Enabled for the Real-Estate Price Prediction System using Machine Learning

9. CONCLUSION AND FUTURE SCOPE

The **Real-Estate Price Prediction System using Machine Learning** provides an efficient and intelligent solution for estimating property prices. By utilizing machine learning algorithms and data-driven approaches, the system overcomes the limitations of traditional manual valuation methods and delivers accurate, reliable, and fast predictions. It helps buyers, sellers, and investors make informed decisions by analyzing key factors such as location, area, BHK, and amenities.

The system's user-friendly interface and use of modern technologies ensure easy accessibility and smooth interaction for users. Real-time prediction, data preprocessing, and model-based analysis enhance the overall performance and usability of the system. The integration of machine learning models such as Linear Regression, Random Forest, and Gradient Boosting improves accuracy and allows the system to handle complex real-estate data effectively.

Looking ahead, the system has great potential for future enhancements. Features such as real-time data integration from real-estate websites, Geographic Information System (GIS) mapping, and mobile application development can further improve usability and accuracy. Advanced techniques like deep learning, AI-based recommendations, and market trend analysis can provide deeper insights and more precise predictions. These improvements will make the system more powerful, scalable, and valuable for real-world applications in the real-estate industry.

Future Scope

1. Integration with Advanced Data Analytics and Machine Learning

The system can be enhanced by using advanced machine learning and deep learning algorithms to improve prediction accuracy. By analyzing large-scale real-estate data, the system can provide better insights, identify market trends, and offer more precise property price estimations.

2. Real-Time Data Integration

- 3. Linking the system with real-estate websites and APIs can enable real-time data collection.**
This will ensure that property prices are updated dynamically based on current market conditions, improving reliability and relevance.

4. Investment and Market Analysis Module

Future versions can include features for investment analysis, helping users identify profitable properties. The system can provide suggestions based on location trends, price growth, and return on investment (ROI).

5. Smart Recommendations and Alerts

The platform can introduce smart recommendations such as suggesting best locations for buying property or alerting users about price changes, market trends, and investment opportunities.

6. AI-Based Virtual Assistant

To improve user experience, future upgrades could include AI-powered assistants that help users input property details, explain predictions, and provide guidance for real-estate decisions.

7. Geographic Information System (GIS) Integration

Integrating GIS and map-based visualization can help users view property locations, nearby facilities, and area-wise price variations, making the system more interactive and informative.

8. Multi-Region and Global Support

The system can be expanded to support international real-estate markets, including multiple currencies and region-based pricing models for global users.

9. Mobile Application Development

Developing a mobile application will allow users to access the system anytime and anywhere, improving accessibility and usability.

10. Advanced Visualization and Dashboard

Future improvements can include interactive dashboards with detailed graphs and comparisons to help users better understand property trends and predictions.

REFERENCES

- [1] Tanguturu Akhil, et al. “Real Estate Price Prediction Using Machine Learning Techniques.” International Journal of Current Science, 2023.
- [2] Yash Panchal, et al. “Residential Property Price Prediction Using Machine Learning (MakanSETU).” International Conference on Emerging Technologies, 2023.
- [3] Vedang Matey, et al. “Real Estate Price Prediction Using Supervised Learning Algorithms.” International Institute of Information Technology (I2IT), Pune, 2022.
- [4] Rounak Banik, et al. “Housing Price Prediction Using Machine Learning Algorithms.” International Journal of Engineering Research & Technology (IJERT), 2022.
- [5] S. K. Pandey and A. Sharma. “A Comparative Study of Machine Learning Algorithms for Property Price Prediction.” International Journal of Computer Applications, 2023.
- [6] J. Brownlee. “Machine Learning Mastery for Predictive Modeling.” Machine Learning Journal, 2022.

Websites:

- <https://www.kaggle.com/>
- <https://scikit-learn.org/>
- <https://www.geeksforgeeks.org/>
- <https://www.analyticsvidhya.com/>
- <https://www.machinelearningmastery.com/>